HW1: Mid-term assignment report

João Miguel Padrão Neves [124834], v2026-10-27

1 Introduction....1
1.1 Overview of the work....1
1.2 My current implementation (limitations & extras)....2
1.3 Use of generative AI....2

2 Product specification....3
2.1 Functional scope and actors....3
2.2 Parcel lifecycle....7
2.3 Business rules....8
2.4 Non-functional requirements....10
2.5 System architecture and design decisions....11
2.6 API for developers....13

3 Quality assurance....14
3.1 Overview of the tests available....14
3.2 Unit tests and test doubles....14
3.3 Integration tests....15
3.4 Functional tests (web automation)....15
3.5 Acceptance tests (BDD)....15
3.6 Non-functional testing....16
3.6.1 Performance....16
3.6.2 Accessibility....17
3.7 Code quality analysis....17
3.8 Continuous integration pipeline [optional]....19

4 References & resources....20

1 Introduction

1.1 Overview of the work

This report presents the midterm individual project required for TQS, covering both the software product features and the adopted quality assurance strategy.
TuttiBox works as a mediator between online shops (partners) and their customers (recipients). It consists of a network of parcel locker stations: when ordering, the recipient chooses a nearby station instead of giving a home address. The partner registers the parcel, the system reserves a compartment, a courier deposits it, and the recipient collects it with a single-use pickup code. An operator supervises each station. This solves problems such as:

- Failed deliveries: the recipient does not need to be at home and can collect the parcel whenever it is convenient.

- Privacy: the recipient does not have to share a home address with third parties, only a public place.

Each station has compartments in up to three sizes (S, M or L), and each parcel is assigned a compartment that fits its size.

1.2 My current implementation (limitations & extras)

All the six use cases are implemented and work end-to-end, with some simplifications. Partners have no web page. They use only the REST API, and the API is documented only in the README (there is no Swagger/OpenAPI). There is no authentication either: the operator pages and the partner endpoints are open to anyone. Stations and compartments are fixed seed data for Aveiro, and there is no way to create or edit them. Opening hours, prices and shipping fees are also out of scope.

The locker hardware is an in-memory fake. In the application, doors always open and are reported closed immediately. Door failures (a door that does not open, or does not close) are only injected in tests. Notifications, such as the pickup code, are written to a log channel instead of being sent by SMS or email.

The 5-day drop-off and pickup deadlines are applied by a scheduled task that runs every minute, so a parcel can change to CANCELLED or EXPIRED up to one minute late. The geocoding cache is kept in memory, so it is lost when the application restarts and is not shared between instances. Tracking times are shown in UTC.

Even so, every external dependency sits behind a port. Connecting real hardware, a real SMS gateway or another geocoding provider would only need a new adapter, without changing the business rules or the tests.

Beyond the base use cases, the application also includes:

- Pickup codes are stored as an HMAC, with the terminal blocked for 2 minutes after 5 wrong codes
- Allocation of the next larger compartment(S → M → L) when the requested size is full
- Safe allocation under concurrent requests
- A full state history of each parcel (who changed it and when)

1.3 Use of generative AI

To accelerate the development process, I used Claude to generate the production code and implement the tests, based on the decisions I had made throughout the project. However, I did not delegate the definitions of the rules, user stories, requirements, or testing strategies to the AI. I wanted to retain control over the application's expected behavior and ensure that all major decisions remained my own.

The only design decision I delegated to the AI was the user interface layout, as I considered it a secondary concern compared to the application's functionality. To generate the initial implementation, I provided Claude with my partially completed report, which was missing the section 3, along with the assignment document. I then asked it to build the project and implement all the required features according to the decisions and specifications described in those documents, allowing it to choose the layout.

After receiving the completed implementation, I ran the application and tested its functionality. However, I was not satisfied with the initial layout, so I asked Claude to make the interface more user-friendly and visually appealing by introducing more

colour. In response, it modified only the HTML and CSS files, leaving the remaining implementation unchanged.
Despite using AI to accelerate development, I still reviewed the generated code and addressed issues that I identified during the process. For example, as discussed later in this report, SonarQube identified an issue in the code responsible for checking whether an address was stored in the cache. The original implementation compared the result of cache.get(address) with null, despite the use of an Optional to represent the possible absence of a value. I addressed this issue by changing the condition to check whether the cache contained the address before attempting to retrieve its value. This made the intended logic more explicit and avoided the unnecessary comparison with null.

2 Product specification

2.1 Functional scope and actors

ID	User story	Acceptance criteria	Acceptance test?
US1	As a recipient, I want to put an address and see the nearest stations, with how many free compartments each one has per size so that I can choose the most convenient locker for me.	Given stations with free compartments exist, When the recipient searches for a valid address, Then the 5 nearest stations are shown ordered by distance, each with its number of free S/M/L compartments.	Playwright
		Given a station with reserved, occupied or out-of-service compartments, When it appears in the results, Then only free, in-service compartments are counted.	No
		Given the geocoding provider returns no result, When the recipient searches for that address, Then the page shows “Address not found” and no stations.	Playwright
		Given the geocoding provider is unavailable or takes longer than 5 s, When the recipient searches, Then the page shows “Service temporarily unavailable”.	No

ID	User story	Acceptance criteria	Acceptance test?
US2	As a partner, I want to register a parcel for a station, giving its size and the recipient's contact, so that a compartment is reserved and I get a tracking code.	Given a station with a free compartment of a compatible size, When the partner registers a parcel, Then it is created in state CREATED, the compartment is reserved and a tracking code is returned (201).	Cucumber
		Given a station with no free compatible compartment, When the partner registers a parcel, Then the request is rejected with 409 and an error body.	Cucumber
		Given a station with only one free compartment, When two partners register a parcel at the same time, Then exactly one succeeds and the other gets 409.	No
		Given an unknown station, invalid size or missing recipient contact, When the partner registers a parcel, Then the request is rejected with 400.	No
		Given a parcel in CREATED, When the partner cancels it, Then it becomes CANCELLED and the compartment is freed.	No
		Given a parcel in any other state, When the partner cancels it, Then the request is rejected with 409.	No
US3	As a courier, I want to put the tracking code and deposit the parcel in the reserved compartment, so that I can leave the parcel available for the recipient and send a pickup code to the recipient	Given a parcel in CREATED, When the courier enters its tracking code and the door is confirmed closed, Then the parcel becomes READY_FOR_PICKUP and a pickup code is sent to the recipient.	Cucumber + Playwright

ID	User story	Acceptance criteria	Acceptance test?
		Given a parcel in CREATED registered more than 5 days ago, When the deadline check runs, Then the parcel becomes CANCELLED and the compartment is freed.	Cucumber
		Given an unknown tracking code or a parcel not in CREATED, When the courier enters it, Then the terminal shows an error and no compartment opens.	No
		Given a parcel in CREATED whose compartment fails to open, When the courier enters the tracking code, Then the compartment is put out of service and the parcel is moved to another free compatible compartment. If none is free, the parcel stays CREATED and the operator is notified	Cucumber
US4	As a recipient, I want to introduce the pickup code and open the right compartment so that I can pick up my parcel and make the compartment available for new reservations	Given a parcel in READY_FOR_PICKUP, When the recipient enters the correct pickup code, Then the compartment opens, the parcel becomes PICKED_UP and the compartment is freed.	Cucumber
		Given a parcel already PICKED_UP, When its pickup code is entered again, Then it is rejected.	Cucumber
		Given 5 consecutive wrong codes at a station, When another code is entered, Then the terminal rejects it for 2 minutes.	Cucumber
		Given a parcel in READY_FOR_PICKUP for more than 5 days, When the deadline check runs, Then the parcel becomes EXPIRED and its pickup code stops working.	Cucumber

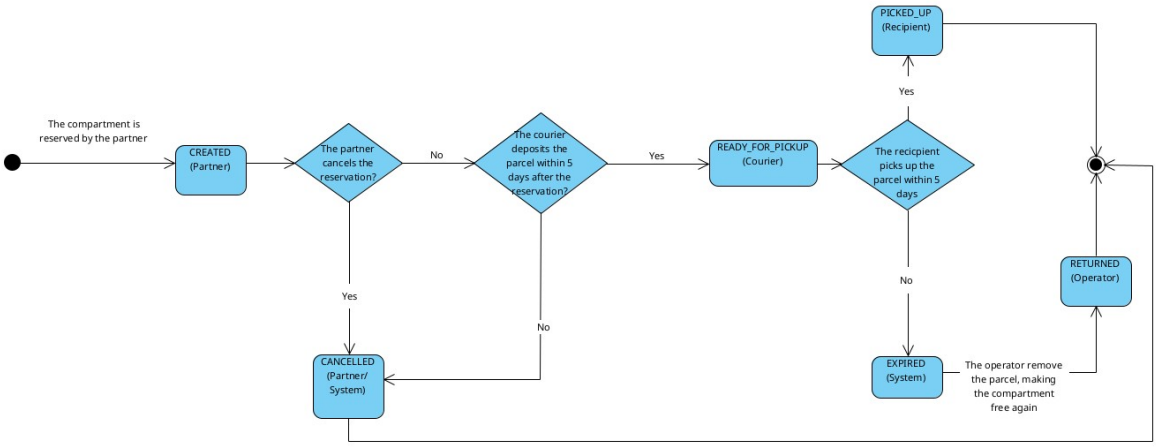
ID	User story	Acceptance criteria	Acceptance test?
		Given a parcel in READY_FOR_PICKUP whose compartment fails to open, When the recipient enters the correct code, Then the parcel stays READY_FOR_PICKUP, the compartment is flagged and the operator is notified.	Cucumber
US5	As a recipient or a partner, I want to track each parcel's state and transitions history with their respective date and time so that I can track its progress up to delivery	Given a parcel with recorded transitions, When the recipient or partner looks up its tracking code, Then the current state and the history (timestamp and actor for each transition) are shown.	No
		Given an unknown tracking code, When it is looked up, Then the response is 404 / "Parcel not found".	No
		Given any parcel, When it is tracked, Then the pickup code is never included in the response.	No
US6	As an operator, I want to supervise each parcel by state, identify expired parcels, register the removal of expired parcels and deactivate non functional compartments and reactivate them once repaired so that I can guarantee a proper functioning of the station	Given a station with parcels in several states, When the operator filters by a state, Then only parcels in that state are listed.	No
		Given a parcel in EXPIRED, When the operator registers its removal, Then it becomes RETURNED and the compartment is freed.	Cucumber
		Given a parcel in any other state, When the operator registers its removal, Then it is rejected with 409 CONFLICT error.	Cucumber
		Given an empty compartment, When the operator puts it out of service, Then it is no longer used for allocation.	No

ID	User story	Acceptance criteria	Acceptance test?
		Given an occupied compartment, When the operator tries to put it out of service, Then the operation is rejected with a 409.	Cucumber
		Given a compartment OUT_OF_SERVICE, When the operator reactivates it, Then it becomes FREE and can be allocated again.	No

2.2 Parcel lifecycle

Parcel state machine diagram

- The following diagram represents all the states a parcel goes through, from registration to delivery

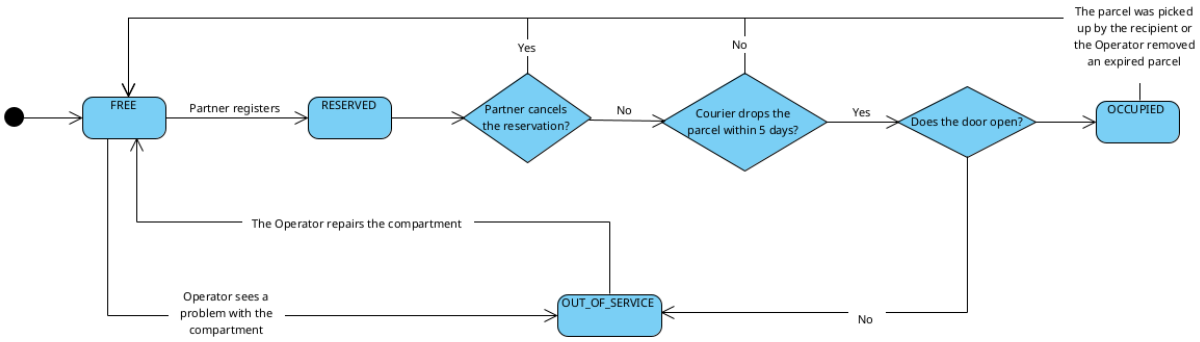


State	Description	Entered by (event, actor)
CREATED	Compartment reserved, waiting for the courier	Registration(Partner)
READY_FOR_PICKUP	Parcel deposited, pickup code sent to the recipient	Drop-off(Courier)
PICKED_UP	Parcel collected, compartment freed(final)	Pickup(Recipient)

CANCELLED	Never deposited, compartment freed(final)	Cancel(Partner) or drop-off deadline(System)
EXPIRED	Not collected in time, pickup code no longer works	Pickup deadline(System)
RETURNED	Removed by the operator, compartment freed(final)	Removal(Operator)

Compartment state machine diagram

- The same was done for the compartment states



2.3 Business rules

Rule/theme	Value/rationale	Configured in	Tested in
Compartment allocation	Same size as the parcel. If there isn't any of that same size, a bigger one because it allows to give a more efficient use of the compartments	application.yml: tuttibox.allocation.allow-larger-size: true	SizeTest ReservationServiceIT register_parcel.feature
Drop-off deadline	5 days between the partner registers the parcel and put it ready to pick up. I chose this because it is closer to the real implementations like InPost or Vinted Go. After that, the parcel becomes CANCELLED and the compartment is freed.	application.yml: tuttibox.deadlines.drop-off: 5d	ReservationServiceIT drop_off.feature
Pickup deadline	5 days between the parcel is ready for pick up and the recipient picks it up, if it passes that period the parcel will be marked as EXPIRED. I chose this because it is closer to the real implementations like InPost(5 days) or Vinted Go(7 days).	application.yml: tuttibox.deadlines.pickup: 5d	ReservationServiceIT pickup.feature

Rule/theme	Value/rationale	Configured in	Tested in
Pickup code (format, validity, protection against guessing)	Random 6 digits code for unique use. Is just valid during the pick up state. After 5 consecutive wrong codes, the station's terminal is blocked for 2 minutes. I chose this because the number of possible combinations is 1 000 000 and with 5 attempts every 2 minutes, an attacker can try at most, (24 × 60 / 2) × 5 = 3,600 codes per day, taking months to just guess a pin.	application.yml: tuttibox.pickup-code.length: 6, max-attempts: 5, lockout: 2m	PickupCodeServiceTest TerminalLockoutTest ReservationServiceIT ApiIT pickup.feature
Hardware failures (one row per failure mode you identified)	If the compartment's door does not open during the drop-off, it will be marked as out of service, the operator is notified and the parcel is moved to another compartment, this solves the unexpected situation quickly by just switching the compartment. If none is free, the parcel stays CREATED and the operator is notified	Not configurable:rules in ReservationService	ReservationServiceIT drop_off.feature
	If the compartment's door does not open or has a malfunction during the pick-up the parcel will keep its state and it will notify the operator, allowing that the parcel will not be lost and solving the situation without involving too much the recipient who doesn't have any fault in the situation.	Not configurable: rules in ReservationService	ReservationServiceIT ApiIT pickup.feature
	If the compartment's door is not confirmed closed after the drop-off, the parcel is not considered deposited: it stays CREATED, no pickup code is sent, and the operator is notified. This way the recipient is never told a parcel is ready when it may not be safely locked	Not configurable: rules in ReservationService	ReservationServiceIT
Search of closest stations	When the recipient puts his address and search for nearby stations, it should show the 5 closest stations to the given address, 5 options are enough to choose a convenient station without a long list.	tuttibox.search.max-results: 5	StationServiceTest ApiIT StationSearchE2E

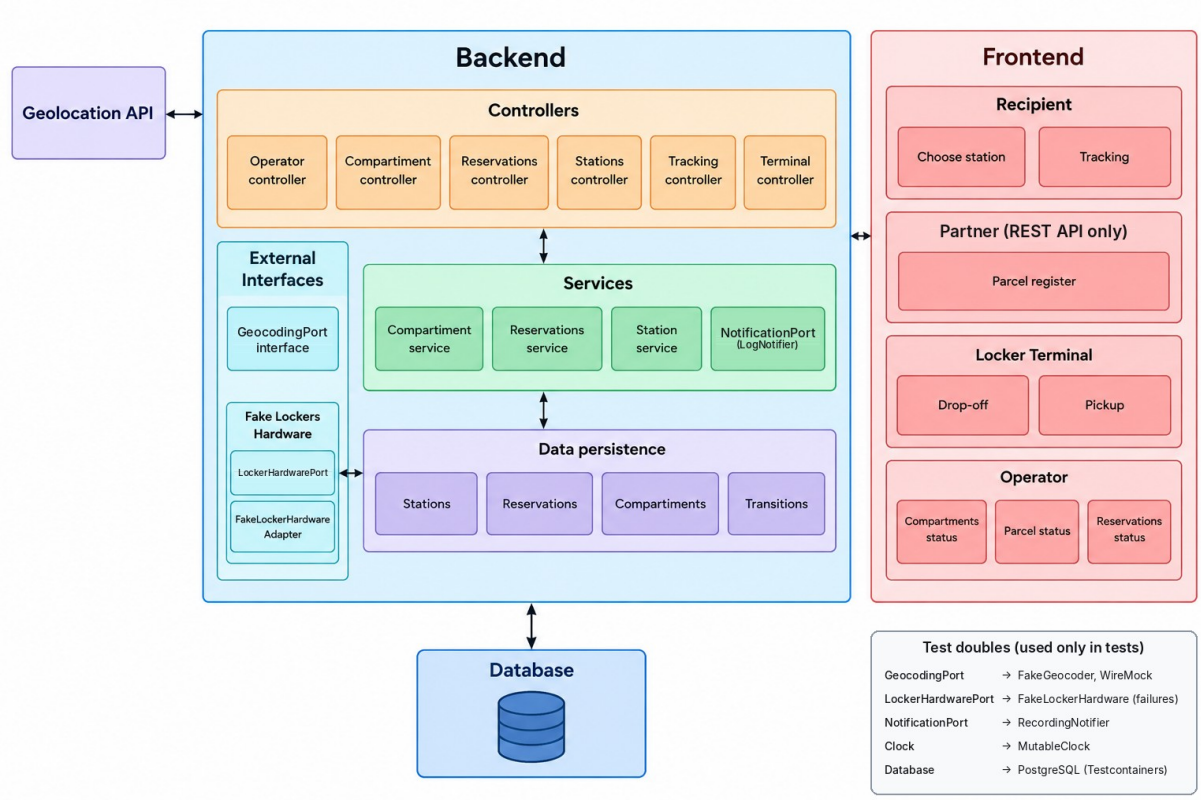
Rule/theme	Value/rationale	Configured in	Tested in
	If the geolocation provider does not find anything for the given address, it should just appear “Address not found” and no nearby stations. I chose this because that’s what the most of the applications do when doesn’t find anything and to warn the user about the return	Not configurable: rule in StationService	StationServiceTest NominatimGeocodingClientIT ApiIT StationSearchE2E
	To avoid waiting forever because the geolocation provider is unavailable it should have a timeout of 5 seconds. I chose this time because it is an average value between a time too short like 1 or 2 and not too big like 10/15 that makes the system more slow	tuttibox.geocoding.timeout: 5s	NominatimGeocodingClientIT ApiIT
Booking compartments by the partner	If the partner tries to book a compartment in a station that it is full or two partners try to book the last free compartment, it should return a 409 CONFLICT error. This code error is what fits better in this situation where the request conflicts with the current state of the station (no free compartment)	Not configurable	AllocationConcurrencyIT ApiIT register_parcel.feature
Remove parcels	The operator can only remove parcels that are already expired or else it will return a 409 CONFLICT error. I chose this error because it is an actual conflict between the operator that wants to remove the parcels and their state that is before EXPIRED, the recipient is still entitled to collect it	Not configurable: transition table in ParcelLifecycle	ReservationServiceIT operator.feature

2.4 Non-functional requirements

ID	Requirement (measurable)	Verification method/tests implemented
NFR1	With 50 concurrent users searching for stations and registering parcels, 95% of the requests must have a time response below 1 second and the error rate must be below 1%. The geocoding provider will be stubbed	Load tests with k6

ID	Requirement (measurable)	Verification method/tests implemented
NFR2	The station search page, the tracking page and the locker terminal have a Lighthouse accessibility score of at least 90 (out of 100).	Manual: It will be tested using the Lighthouse browser tool. This too will give a grade for the accessibility.
NFR3	The pickup code has 6 random digits, single use, valid only while READY_FOR_PICKUP, stored only as an HMAC hash, never returned by the API or written to the application log (it only leaves the system through the notification channel, which simulates the SMS). 5 wrong attempts block the terminal for 2 minutes.	Unit test: PickupCodeServiceTest TerminalLockoutTest Integration tests: ReservationServiceIT ApiIT
NFR4	According to the geocoding provider rules, we can only have 1 request per second, every request sends an identifying User-Agent, the same address is never sent twice within a short interval: answers are cached, failed lookups are remembered for 60 s, and concurrent searches for the same address share one request. The OSM attribution is shown on the search page. The provider can be turned off by configuration.	Automated tests with WireMock simulating Nominatim (NominatimGeocodingClientIT): 1 request/s, User-Agent, cache, failure window, concurrent searches. OSM attribution: checked manually on the search page GeocodingConfigurationTest: provider turned off by configuration.

2.5 System architecture and design decisions



Here I have the application's architecture. The backend uses Spring Boot and the frontend uses Thymeleaf pages, a direct choice since the main goal is implementing the requirements and testing them. Data is stored in a PostgreSQL database.

Addresses are converted to geographic coordinates with the OSM Nominatim API, and the locker hardware (opening and closing doors) is simulated by a fake adapter, FakeLockerHardware.

Design decision	Your option	Testing strategy/impact
Simulation of the locker terminal	*A Thymeleaf web page that will play as the touchscreen: the courier enters a tracking code and the recipient enters a pickup code. The page never talks directly to the hardware, the backend does through the hardware interface*	*The terminal journeys are automated with Playwright.*
Representation of the parcel lifecycle	*An enum of states (ParcelState) plus a transition table (ParcelLifecycle) that maps (state, event) to the next state; any event not in the table is rejected. Every transition is stored with its actor and timestamp (Transition), which is what tracking shows.*	*Unit test is enough to verify the several parcel behaviors. Invalid transitions are checked to return 409 in integration tests.*
Controllable time in tests	*A java.time.Clock will be injected into the services. No code calls now() directly*	*Tests will be using a fixed clock and move it forward 5 days to trigger events like CANCELLED or EXPIRED without waiting or sleeping. Used in integration tests and in the Cucumber time-based scenario. In the unit tests the domain classes receive the time as a parameter*
Hardware port: operations; synchronous or event-driven	*Synchronous port with two operations: openDoor() and isDoorClosed(). The adapter is FakeLockerHardware*	*The fake can be configured by test to fail, so the failure rules are tested deterministically, without timing issues.*
Concurrency in compartment allocation	*Only FREE compartments are considered. The compartment is locked while it is being reserved (`FOR UPDATE SKIP LOCKED`), so two reservations at the same time always get different compartments.*	*An integration test with Testcontainers (real PostgreSQL) sends parallel requests for the last free compartment and checks that exactly one succeeds and the others get an error NO_FREE_COMPARTMENT that after will be taken by the API and return a 409 error.*
Pickup codes: generation, storage, checking	*It will be a random 6 digit number from SecureRandom, stored as an HMAC, single use, valid only while READY_FOR_PICKUP. 5 wrong attempts and the station's terminal is blocked for 2 minutes*	*Unit tests for format, randomness, hashing and the lockout counter. Integration and Cucumber tests for single use, expiry and the 2-minute block (with the controllable Clock). The integration tests check that the plain code never appears in the API response or the database.*
Geocoding provider: choice; behavior when slow, unavailable or over limits	*OSM Nominatim behind a GeocodingPort interface: 5 s timeout, at least 1 s between requests, and the same address is never sent twice in a short interval (cache). If it is slow, down or over the limit, the page shows 'Service temporarily unavailable'; an empty result shows 'Address not found'.*	*The HTTP client is tested against WireMock (found, empty, slow, 5xx, 429, cache, rate limit). The ranking logic is unit-tested with a Mockito mock. The real API is never called in the build or in load tests.*

Design decision	Your option	Testing strategy/impact
REST API design: resources, status codes, error format	*REST resources under /api: stations (list, nearest), parcels (register, track, cancel), terminal (drop-off, pickup), operator (parcels by state, removal) and compartments (out of service, reactivation). Status codes: 200, 201 (registration), 400 (invalid request or wrong code), 404 (not found), 409 (business conflict), 429 (terminal blocked), 503 (hardware or geocoding unavailable). Every error has the same JSON body: { status, error, message }.*	*ApiIT (MockMvc, full Spring context) check the status code and error body of every endpoint.*

2.6 API for developers

Method	endpoint	Return
GET	/api/stations	All stations, with free compartments per size
GET	/api/stations/nearest?address=	The 5 nearest stations to the address
POST	/api/parcels	Registers a parcel → tracking code and compartment (201)
GET	/api/parcels/{trackingCode}	Parcel state and transition history
POST	/api/parcels/{trackingCode}/cancel	Cancels a parcel still in CREATED
POST	/api/terminal/{stationId}/drop-off	Courier deposits a parcel (body: tracking code)
POST	/api/terminal/{stationId}/pickup	Recipient collects a parcel (body: pickup code)
GET	/api/operator/stations/{stationId}/parcels?state=	Parcels of a station, optionally filtered by state
POST	/api/operator/parcels/{trackingCode}/removal	Registers removal of an EXPIRED parcel → RETURNED
GET	/api/stations/{stationId}/compartments	Compartments of a station and their state
POST	/api/compartments/{compartmentId}/out-of-service	Puts a FREE compartment out of service
POST	/api/compartments/{compartmentId}/reactivation	Reactivates an OUT_OF_SERVICE compartment → FREE

Errors always return { "status", "error", "message" } with 400, 404, 409, 429 or 503, as described in section 2.5.

3 Quality assurance

3.1 Overview of the tests available

Tests are organized by level, each in its own package under src/test/java:

- Unit tests(domain,service,adapter): business rules without Spring or a database
- Integration tests(it): full Spring context against a real PostgreSQL started by Testcontainers
- Acceptance BDD (bdd.feature in src/test/resources/features): Cucumber scenarios in Gherkin
- Functional/E2E: Playwright browser tests
- Load(load-tests/): k6 scripts

Command	Runs	Docker
mvn test	unit tests	No
mvn verify	unit + integration + Cucumber, and the JaCoCo report	Yes
mvn verify -Pe2e	all of the above + Playwright	Yes
k6 run load-tests/search-and-register.js	load test (app running with the loadtest profile)	Yes

3.2 Unit tests and test doubles

The unit tests cover the domain rules, which are pure Java with no Infrastructure. The parcel's state machine is tested with a parameterized test over the whole transition table as shown in ParcelLifecycleTest: every valid (state, event) pair, every invalid one, and that final states accept no event. Other examples are compartment size compatibility in SizeTest, the distance calculation in GeoMathTest, the 5-attempt lockout in TerminalLockoutTest and the pickup code format and hashing in PickupCodeServiceTest. Domain classes receive the time as a parameter, so time rules need no clock.

The following fakes were created:

- FakeLockerHardware(hardware port):keeps the state of each door (open/closed) and it can inject two failures per compartment, fail to open and never closes. Without failures it behaves as a working locker (demo mode).
- FakeGeocoder(Geocoding port): a map of known addresses→Coordinates. It can be switched to "unavailable" if there isn't any translation
- RecordingNotifier (Notification port): records every notification sent,so tests can read the pickup code the recipient would receive
- MutableClock: a clock that tests move forward

Mocks were used only in StationServiceTest with Mockito for the station ranking. There only fixed answers were needed (coordinates for an address, a list of stations), with no state or behavior to simulate, so a mock was simpler than a fake.

3.3 Integration tests

Integration tests use the full Spring context(@SpringBootTest) rather than test slices: the rules cross several layers (Service,repositories,Transactions,database locks), and a slice would leave out exactly what we want to check. They run against PostgreSQL in Testcontainers, the same engine as production, with the fakes plugged into the ports. The state is reset before each test.

- API: ApiIT uses MockMvc to check status codes and the error body (201,400,404,409,429,503)
- Database and business rules: ReservationServiceIT covers registration, drop-off, pickup, deadlines (with MutableClock), hardware failures and operator actions
- Concurrency: AllocationConcurrencyIT runs 10 threads registering at the same time, with one free compartment, only one of the threads succeeds. It needs a real PostgreSQL because the lock is database feature
- Geocoding client: NominatimGeocodingClientIT runs against a WireMock server that stands in for Nominatim(found, empty,slow,5xx,429,cache,User-Agent,1 request per second). The real API is never called.

3.4 Functional tests (web automation)

Two journeys are automated with Playwright:

- Station search: searching an address shows the 5 nearest stations, and an unknown address shows ”Address not found”.
- Locker terminal: the courier deposits a parcel, the recipient collects it with the code received, and the tracking page shows PICKED_UP. A wrong pickup code is rejected as shown in TerminalJourneyE2E.

These were chosen because they are the main user journeys, one on web pages and one on the terminal, and they cross the whole system from the browser to the database.

Stability: locators use labels, roles and data-testid attributes instead of CSS or position. Playwright’s assertions wait automatically, so there are no sleeps. Each test creates its own data (station and parcel) and resets the fakes, so tests do not depend on each other.

3.5 Acceptance tests (BDD)

Four features, one per use case, with 14 scenarios:

- register_parcel.feature: US2, 3 scenarios
- drop_off.feature: US3, 3 scenarios
- pickup.feature: US4, 5 scenarios
- operator.feature: US6, 3 scenarios

They include time-based scenarios (drop-off and pickup deadlines, moving the MutableClock 5 days forward) and hardware-failure scenarios (a door that does not open at drop-off and at pickup, using FakeLockerHardware).

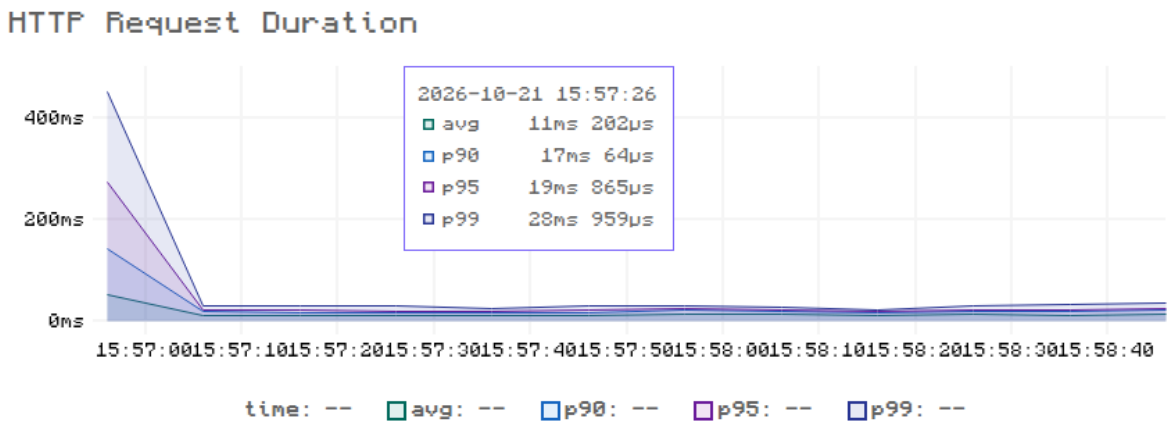
The step definitions (LockerSteps) drive the service layer. It is where the business rules live, the scenarios stay fast and stable, and the HTTP details are already covered by ApiIT and the UI by Playwright.

3.6 Non-functional testing

The load test, written with k6, simulates 50 concurrent users for 2 minutes. Each user searches for the nearest stations, registers a parcel and cancels it, with a 1 second pause between iterations. The application ran locally with the loadtest profile, so the geocoding provider was stubbed and Nominatim was never called. Thresholds encode NFR1: p(95) < 1000 ms and http_req_failed < 1%. When a station is full and it returns a 409 CONFLICT error, it counts as a valid answer.

Metric	Threshold	Result
p95 response time	< 1000 ms	25.3 ms
p99 response time	-	98.1 ms
Error rate	< 1%	0 % (0 /17270)
Throughput	-	142 req/s

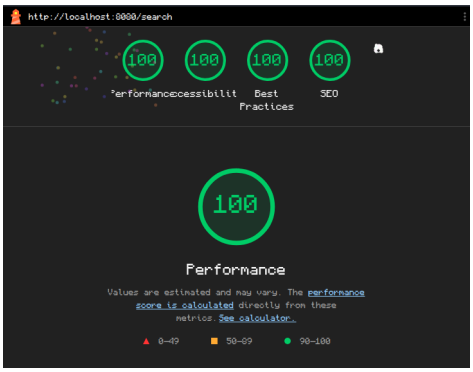
3.6.1 Performance

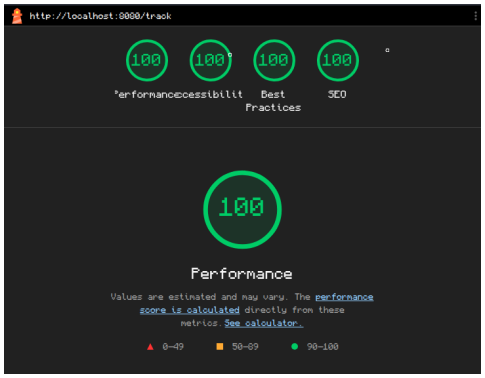


According to the chart above, obtained with k6, and the non-functional requirements, we can say that the NFR1 is met with a large margin, p95 of 25 ms against the 1000 ms limit and no errors. There's a spike at the start during the first 10 s while the JVM warms up, after that, the response time stays stable at around 20 ms until the end, so performance does not degrade over time.
However, throughput stayed at 142 req/s because each user waits 1s between iterations, so the server was never saturated. The tests show that the system handles 50 users, not its maximum capacity, finding the limit would need a stress test with an increasing number of users.

3.6.2 Accessibility









The three pages in NFR2 were audited with Lighthouse (Chrome DevTools). All three scored 100 in accessibility, above the required 90, so there were no issues to fix. The pages were built with accessibility in mind: every input has a <label>, the pages declare their language, the terminal sections are linked to their titles with aria-labelledby, and each page has an <h1>.
A manual keyboard-only walkthrough was done on the search journey (typing an address and reaching the results) and on the terminal (drop-off and pickup forms). Every field and button could be reached, the focus was always visible, and the tab order followed the visual order of the pages.
Lighthouse cannot judge whether the tab order makes sense, whether the focus is always visible, or whether messages are clear to a screen-reader user, that's why the manual walkthrough was needed.

3.7 Code quality analysis

The code quality analysis was made with SonarQube (Community Build, running in Docker) for static analysis, and JaCoCo for test coverage. JaCoCo generates the coverage report during mvn verify, and SonarQube reads it during the analysis

- mvn clean verify sonar:sonar -Dsonar.projectKey=tuttibox -Dsonar.host.url=http://localhost:9000 -Dsonar.token=TOKEN

Quality Gate. The default gate ("Sonar way") only checks the new code, so on a first analysis it always passes. A custom gate, Tuttibox, was defined on the overall code



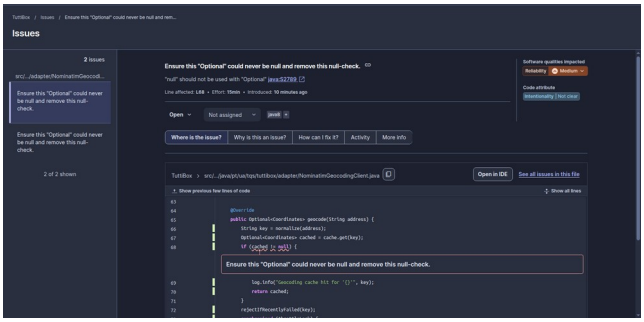
Conditions

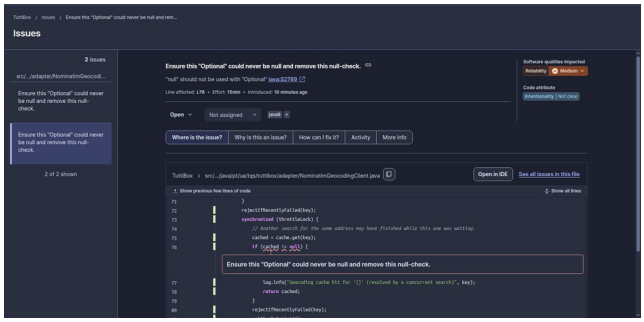
Conditions on Overall Code

Metric	Operator	Value
Coverage	is less than	75.0%
Duplicated Lines (%)	is greater than	3.0%
Issues	is greater than	0
Maintainability Rating	is worse than	A
Reliability Rating	is worse than	A
Security Hotspots Reviewed	is less than	100%
Security Rating	is worse than	A

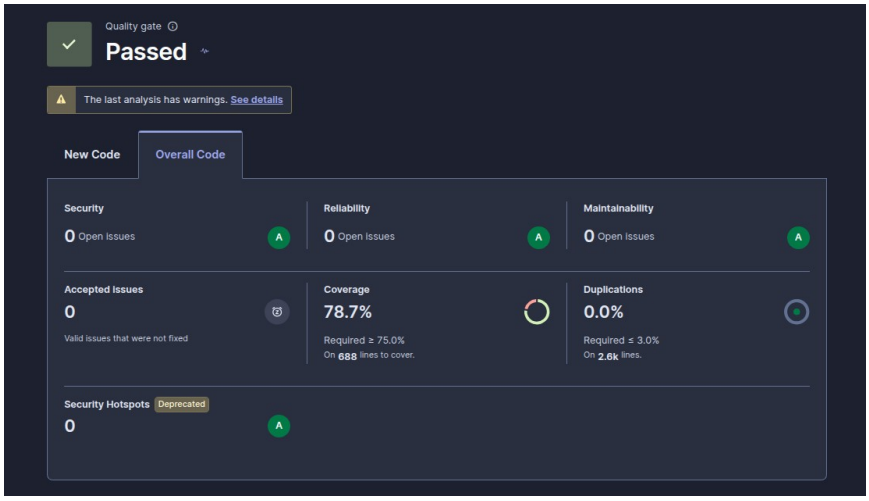
Condition	Fails if	Why
Coverage	< 75%	Most of the code must be tested, without forcing artificial tests just to reach 100%
Duplicated lines	> 3%	Duplicated code means a fix has to be repeated in several places
Issues	> 0	Issues are fixed or reviewed as they appear, so none accumulate
Reliability / Security / Maintainability rating	Worse than A	No known bugs, vulnerabilities or relevant technical debt
Security hotspots reviewed	< 100%	Every security-sensitive point must be checked by a person

After running the tests and register on SonarQube it detected 2 Issues(rule java:S2789, “null should not be used with Optional”) in NominatimGeocodingClient:





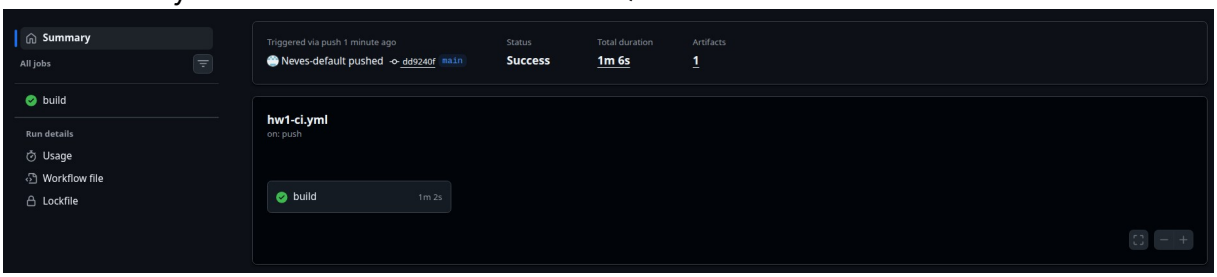
The Geocoding cache used null to mean “”address not in the cache” and Optional to mean “found/not found”, mixing two meanings in the same variable. It was fixed by checking cache.containsKey(key) instead. The behavior is the same and all tests still pass.
After the fix, the gate passed with 0 issues, A rating, 0.0% duplication and 78.7% coverage



The coverage of the web package (Thymeleaf page controllers) is low and was not fixed in the default build, because those pages are tested by Playwright, which only runs with -Pe2e. No extra tests were written just to raise the percentage, since the journeys are already covered in the browser.
With this the main lesson that I learned was SonarQube found a clarity problem nobody had noticed even when the code worked just fine and every test passed. Static analysis catches issues that tests cannot, because tests check behaviour, not how readable the code is.

3.8 Continuous integration pipeline [optional]

A GitHub Actions workflow runs mvn verify on every push and pull request that changes the project. GitHub runners have Docker, so Testcontainers works as it does locally. The JaCoCo report is kept as a build artifact. The Quality Gate is not enforced in CI, because SonarQube runs locally. To enforce it would need SonarQube Cloud.



4 References & resources

Project resources

Resource	URL/location
Git repository	https://github.com/Neves-default/HW1-124834
Video demo	Under the sub-folder /docs
API documentation	Section 2.6 of this report and README.md (REST API summary).

Resource	URL/location
QA dashboard (online)	Local SonarQube (not online). The results are in section “Code Quality analysis”
CI/CD pipeline	https://github.com/Neves-default/HW1-124834/actions

Reference materials

- Nominatim Usage Policy: https://operations.osmfoundation.org/policies/nominatim/
- Spring Boot: https://docs.spring.io/spring-boot/
- Testcontainers: https://java.testcontainers.org/
- WireMock: https://wiremock.org/docs/
- Cucumber: https://cucumber.io/docs/
- Playwright for Java: https://playwright.dev/java/
- k6: https://grafana.com/docs/k6/latest/
- SonarQube Community Build: https://docs.sonarsource.com/sonarqube-community-build/